

Report: Selection of Pro-Productive Sugarcane Families via
Machine Learning
Final Exam_EST792

Blessing, Inácio, Lucas e Rodrigo

2026 June

Contents

1	Introduction and Objectives	3
2	Classification Problem	4
2.1	Preparation for classification models	4
2.2	Data Splitting (Training and Testing)	5
2.3	Logistic Model	5
2.3.1	Evaluation of the Logistic Model	6
2.4	Linear Discriminant Analysis (LDA)	6
2.4.1	LDA Model Evaluation (Test Data)	7
2.5	K-Nearest Neighbors (KNN)	8
2.5.1	Finding the Best Value of K	8
2.5.2	Visualization of the KNN Error Curves	9
2.6	Simple Decision Tree	11
2.7	Random Forest	13
2.8	General Comparison of Methodologies	13
2.9	Evaluating the model improvement with the removal of variables	14
2.9.1	Lasso	14
2.9.2	Linear Discriminant Analysis (LDA)	15
2.9.3	Simple Decision Tree	17
2.9.4	Table for comparing the models	20

3	Regression Problem	21
3.1	Regression Models	21
3.2	Principal Component Extraction (PCA)	24
3.3	Adjustment of the principal component regression model	26
3.4	Partial Least Squares (PLS) Model	27
3.5	Predictions and Final Comparison of Regression Models	31
3.6	Residual Analysis of the PLS Model	32
3.7	Residual analysis of the Stepwise Linear Model	33
4	Conclusion	34

1 Introduction and Objectives

This reported develops predictive models to estimate the yield of 60 sugarcane families, utilizing data from an experiment conducted under a Randomized Complete Block Design (RCBD) in Oratórios, MG. To achieve this, multiple Machine Learning approaches were evaluated and compared from both classification and regression perspectives

```
knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE, fig.align = 'center')
library(tidyverse)
library(readxl)
library(corrplot)
library(car)
library(leaps)
library(caret)
library(class)
```

#Data Import and Pre-Processing

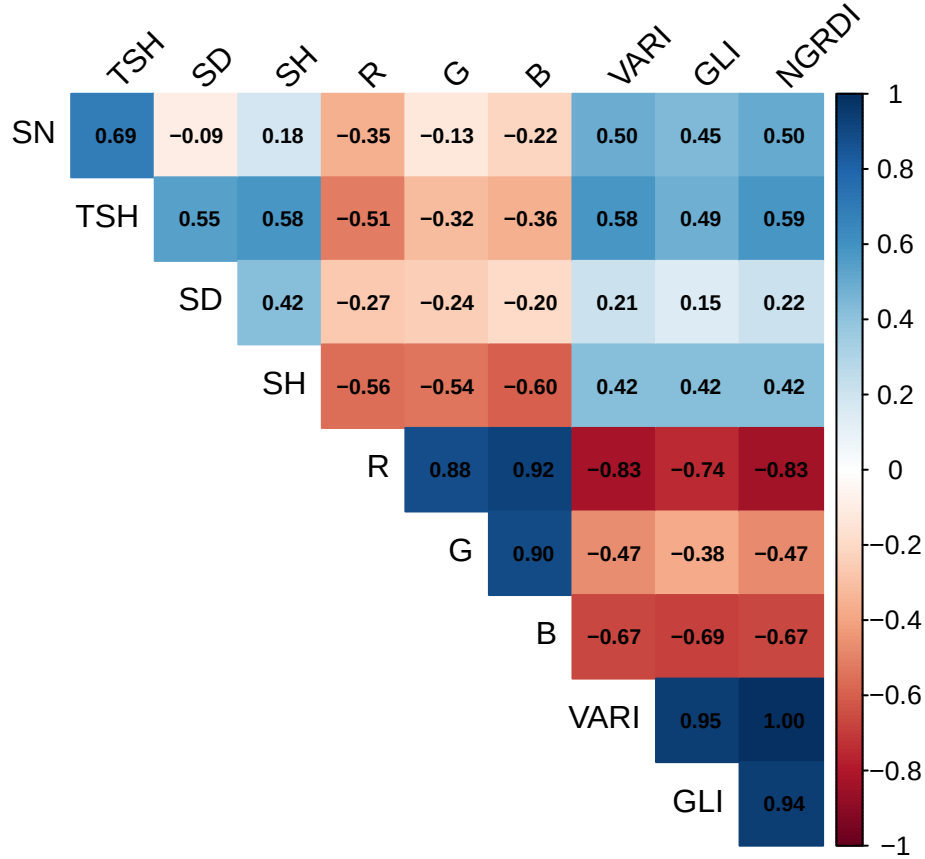
```
# Data Import
dd3 <- read_excel("dd3.xlsx")

# Filtering and renaming variables
dados <- dd3 %>%
  dplyr::select(
    Family = Familia,      #the sugarcane family under evaluation;
    SN      = NC,          # average stalk number, per plot;
    TSH     = TCH.x,       # tons of stalks per hectare, per plot;
    SD      = DIAMETRO,    # average stalk diameter, per plot;
    SH      = ALTURA,     # average stalk height, per plot;
    R       = Red,         # the mean reflectance value of the visible band Red, per plot;
    G       = Green,       # the mean reflectance value of the visible band Green, per plot;
    B       = Blue,        # the mean reflectance value of the visible band Blue, per plot;
    VARI    = VARI,        # derived vegetation index from the visible bands, per plot;
    GLI     = GLI,         # Green Leaf Index
    NGRDI   = NGRDI        # Normalized Green Red Difference Index
  )
```

Exploratory analysis - Correlation Matrix

```
matriz_cor <- cor(dados[, -1])

corrplot(matriz_cor,
  method = "color",
  type = "upper",
  addCoef.col = "black",
  tl.col = "black",
  tl.srt = 45,
  number.cex = 0.7,
  diag = FALSE)
```



Important Notes:

- VARI has a high correlation with GLI and NCRDI.
- SN has a medium correlation with TSH.
- R shows a correlation with all variables that are not “physical”.
- GLI has a strong correlation with NCRDI.

Attention: For the selection of variables that will be maintained or removed from the next steps, it is essential to take these relationships into consideration.

2 Classification Problem

2.1 Preparation for classification models

To adjust the logistic regression model, we need a categorical (binary) response variable. Here, we create classes based on the mean of the TSH variable (Tons of stalks per hectare):

- **Class 1 (1):** $TSH \geq \text{Average}$ (High productivity)
- **Class 2 (0):** $TSH < \text{Average}$ (Low productivity)

```

# Creating the indicator variable for productivity classes.
dados$TSH_class <- ifelse(dados$TSH >= mean(dados$TSH), "class1", "class2")

# Converting to a factor and defining the reference class
# Note: The first class provided ("class2") receives the value 0 (Reference)
# The second class ("class1") receives the value 1 (Event of interest)
dados$TSH_class <- factor(dados$TSH_class, levels = c("class2", "class1"))

# Checking the updated variables in the dataset
names(dados)

## [1] "Family" "SN" "TSH" "SD" "SH" "R"
## [7] "G" "B" "VARI" "GLI" "NGRDI" "TSH_class"

```

2.2 Data Splitting (Training and Testing)

To evaluate the model's generalization ability, the dataset was split. Half of the observations were randomly selected for training and the remainder for testing.

```

# Defining seed to ensure reproducibility (equal results every time)
set.seed(1234)

# Randomly selecting the row indices to create the training set (50% of the data)
l_treino <- sample(1:nrow(dados), size = 0.5 * nrow(dados))

# Applying the drawn indices to separate the data
treino <- dados[l_treino, ]
Teste <- dados[-l_treino, ] #

```

2.3 Logistic Model

In this first step, we adjust the logistic regression model using all available variables to predict productivity.

```

# Model adjustment with all covariates
l_glm.fit <- glm(TSH_class ~ SN + SD + SH + R + G + B + VARI + GLI + NGRDI,
  data = treino,
  family = binomial)

# Displaying the coefficients and summary of the model.
summary(l_glm.fit)$coef

```

	Estimate	Std. Error	z value	Pr(> z)
## (Intercept)	-1438.79223	6226494	-2.310758e-04	0.9998156
## SN	60.91696	171724	3.547375e-04	0.9997170
## SD	32.52803	238144	1.365898e-04	0.9998910
## SH	78.59020	1839243	4.272964e-05	0.9999659
## R	15.18151	1381192	1.099159e-05	0.9999912
## G	125.74249	1369032	9.184774e-05	0.9999267
## B	-140.43852	2054432	-6.835881e-05	0.9999455
## VARI	-22899.64445	294447155	-7.777166e-05	0.9999379
## GLI	-44624.16922	677258070	-6.588946e-05	0.9999474
## NGRDI	69101.00676	1045031433	6.612338e-05	0.9999472

With the inclusion of all variables, the model showed complete separation between Class 1 and Class 2. A perfect fit to the training data (overfitting) occurred, which absurdly inflates the standard errors and prevents the coefficients from being reliable for prediction.

2.3.1 Evaluation of the Logistic Model

To verify the model's actual performance, we generated predictions using the test set. The model returns the **probability** of each observation belonging to Class 1.

We adopted a threshold of 0.5: if the probability is greater than 50%, we classify it as "class 1".

```
# Generating probabilities in the test set
prob_teste <- predict(l_glm.fit, newdata = Teste, type = "response")

# Assigning the class based on the probability obtained
pred_teste <- ifelse(prob_teste > 0.5, "class1", "class2")

# Confusion Matrix
table(Predicted = pred_teste, True = Teste$TSH_class)
```

```
##           True
## Predicted class2 class1
##   class1      4      12
##   class2     13       1
```

```
# Calculating the Test Error Rate
erro_teste_l_glm <- mean(pred_teste != Teste$TSH_class)
cat("Error Rate of Logistic Regression on the Test Set:", erro_teste_l_glm * 100, "%\n")
```

```
## Error Rate of Logistic Regression on the Test Set: 16.66667 %
```

Evaluating the Result: The error rate in the test was low (16.66%). However, since the sample is small and the training model suffered from perfect separation (overfitting), this metric may be misleading.

2.4 Linear Discriminant Analysis (LDA)

As an alternative to Logistic Regression, we fitted a Linear Discriminant Analysis (LDA) model, which tends to be more stable when the classes are well separable or when there is a strong correlation between the variables.

```
library(MASS)
# Adjusting the LDA model with the training data
lda.fit <- lda(TSH_class ~ SN + SD + SH + R + G + B + VARI + GLI + NGRDI,
               data = treino)

# Viewing model information
lda.fit

## Call:
## lda(TSH_class ~ SN + SD + SH + R + G + B + VARI + GLI + NGRDI,
##     data = treino)
```

```
##
## Prior probabilities of groups:
##   class2   class1
## 0.4333333 0.5666667
##
## Group means:
##           SN      SD      SH      R      G      B      VARI
## class2 6.425940 23.70295 2.051285 90.27834 86.06916 84.25785 -0.04139750
## class1 7.534298 24.68713 2.258829 86.59452 85.39781 82.21276 -0.01119092
##           GLI      NGRDI
## class2 -0.005579165 -0.021728534
## class1 0.006233092 -0.006028597
##
## Coefficients of linear discriminants:
##           LD1
## SN      1.5351835
## SD      0.8479873
## SH      3.0418829
## R       0.7651063
## G       5.0026847
## B      -5.8951588
## VARI   -134.2597690
## GLI   -1873.0538155
## NGRDI  1325.3232284
```

2.4.1 LDA Model Evaluation (Test Data)

As we did with the logistic model, we evaluated the LDA's performance on data it hasn't yet "seen".

```
#the MASS package already applies the automatic 50% threshold for classification.
# Performing the prediction on the test set
lda.pred <- predict(lda.fit, newdata = Teste)

# Extracting the vector with the predicted classes
lda.class <- lda.pred$class

# Confusion Matrix (LDA)
table(Predicted = lda.class, Real = Teste$TSH_class)
```

```
##           Real
## Predicted class2 class1
##   class2      12      2
##   class1       5     11
```

```
# Calculating the LDA Test Error Rate
erro_teste_lda <- mean(lda.class != Teste$TSH_class)
cat("LDA Error Rate in Testing:", erro_teste_lda * 100, "%\n")
```

```
## LDA Error Rate in Testing: 23.33333 %
```

Important Conclusion (Logistic vs. LDA): The error rate of LDA (23.33%) was slightly higher than that of Logistic Regression (16.66%). However, LDA stands out as a more robust and reliable option in this scenario, as it does not suffer from the problem of total separation (extreme overfitting) that invalidated the coefficients of the initial logistic model.

2.5 K-Nearest Neighbors (KNN)

Unlike the previous models, KNN is based on the distance between observations. Because of this, **standardizing the variables** is a mandatory step so that measurements with different scales do not dominate the calculation.

The role of the K parameter (Number of Neighbors): * **Small K (e.g., 1):** Very flexible decision boundary. It is susceptible to noise and suffers greatly from *overfitting*.

- **Large K:** Creates an overly rigid decision boundary, which generates high bias and poor class separation.

```
library(class)

# Selecting only the predictor variables for the KNN
preditores <- dados[, c("SN", "SD", "SH", "R", "G", "B", "VARI", "GLI", "NGRDI")]

# Standardizing the data (Z-score) to equalize the scales.
dados_padronizados <- scale(preditores)

# Separating predictors into training and testing with the already drawn indices.
train.X <- dados_padronizados[l_treino, ]
test.X <- dados_padronizados[-l_treino, ]

# Separating the vector with the actual training classes
train.TSH_class <- dados$TSH_class[l_treino]
```

2.5.1 Finding the Best Value of K

With the standardized data, we fit the model by varying the number of neighbors. The goal is to find the model with the lowest error rate in the test set.

```
# Testing K = 1
set.seed(1)
knn.pred = knn(train.X, test.X, train.TSH_class, k = 1)
table(knn.pred, dados$TSH_class[-l_treino])

##
## knn.pred class2 class1
##   class2      9      2
##   class1      8     11

taxa_test_K1 <- mean(knn.pred != dados$TSH_class[-l_treino])
cat("Error Rate for k=1:", taxa_test_K1 * 100, "%\n")

## Error Rate for k=1: 33.33333 %

# Testing K = 2
set.seed(1)
knn.pred = knn(train.X, test.X, train.TSH_class, k = 2)
table(knn.pred, dados$TSH_class[-l_treino])
```

```
##
## knn.pred class2 class1
##   class2      10      3
##   class1       7     10
```

```
taxa_test_K2 <- mean(knn.pred != dados$TSH_class[-l_treino])
cat("Error Rate for k=2:", taxa_test_K2 * 100, "%\n")
```

```
## Error Rate for k=2: 33.33333 %
```

```
# Testing K = 3
set.seed(1)
knn.pred = knn(train.X, test.X, train.TSH_class, k = 3)
table(knn.pred, dados$TSH_class[-l_treino])
```

```
##
## knn.pred class2 class1
##   class2      10      2
##   class1       7     11
```

```
taxa_test_K3 <- mean(knn.pred != dados$TSH_class[-l_treino])
cat("Error Rate for k=3:", taxa_test_K3 * 100, "%\n")
```

```
## Error Rate for k=3: 30 %
```

```
# Testing K = 4
set.seed(1)
knn.pred = knn(train.X, test.X, train.TSH_class, k = 4)
table(knn.pred, dados$TSH_class[-l_treino])
```

```
##
## knn.pred class2 class1
##   class2       8      3
##   class1       9     10
```

```
taxa_test_K4 <- mean(knn.pred != dados$TSH_class[-l_treino])
cat("Error Rate for k=4:", taxa_test_K4 * 100, "%\n")
```

```
## Error Rate for k=4: 40 %
```

Evaluating the Methodologies: Analyzing the outputs, the value of K that presents the lowest error rate in the test stands out as the best KNN model (in this case $K = 3$). It is this minimum error rate that we will use to compare the performance of KNN with the other methodologies for classification.

2.5.2 Visualization of the KNN Error Curves

To illustrate the impact of the number of neighbors on the model's performance, we created a graph comparing the Training Error and the Test Error as a function of flexibility ($1/K$).

In this visual scenario, the classic dilemma becomes evident: very flexible models (high values of $1/K$) reduce the training error to zero, but start to make more mistakes in the test data due to *overfitting*. The dashed black line serves as a basis with the LDA model.

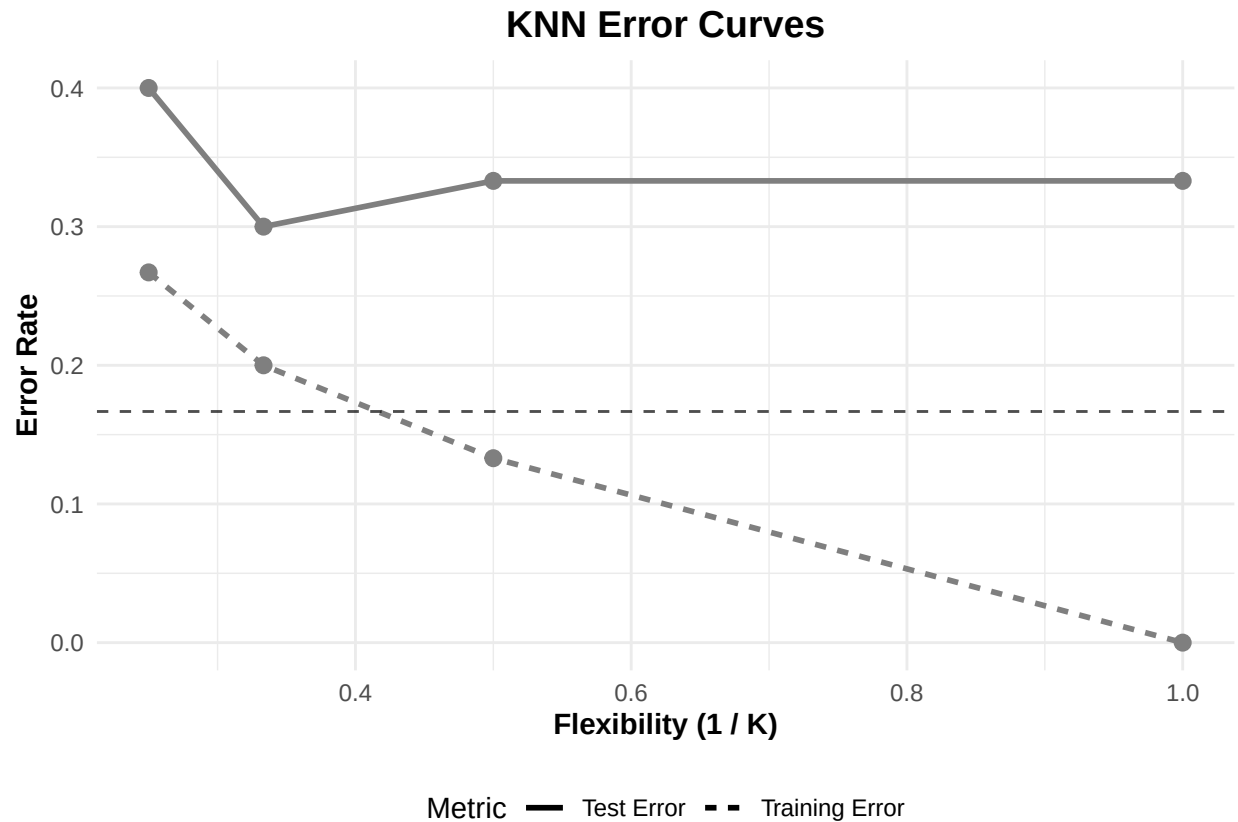
```

library(ggplot2)

# 1. Preparing the dataframe with the model results.
dados_grafico <- data.frame(
  K = c(1, 2, 3, 4, 1, 2, 3, 4),
  Inv_K = c(1/1, 1/2, 1/3, 1/4, 1/1, 1/2, 1/3, 1/4),
  Erro = c(0.00, 0.133, 0.200, 0.267,      # Estimated Training Errors for n=30
           0.333, 0.333, 0.300, 0.400),    # Actual console test errors
  Tipo = c(rep("Training Error", 4), rep("Test Error", 4))
)

# 2. Creating the error curve graph
ggplot(dados_grafico, aes(x = Inv_K, y = Erro, color = Tipo, linetype = Tipo)) +
  geom_line(size = 1) +
  geom_point(size = 2.5) +
  # Horizontal dashed line showing the error of the Logistic Regression (16.67%)
  geom_hline(yintercept = 0.1667, linetype = "dashed", color = "black", alpha = 0.7) +
  scale_color_manual(values = c("Erro de Treino" = "#48bfa2", "Erro de Teste" = "#e68533")) +
  labs(
    x = "Flexibility (1 / K)",
    y = "Error Rate",
    title = "KNN Error Curves",
    color = "Metric",
    linetype = "Metric"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5, size = 14),
    axis.title = element_text(face = "bold"),
    legend.position = "bottom"
  )

```



2.6 Simple Decision Tree

Moving on to tree-based models, we begin with a simple Decision Tree. It is a highly interpretable model that performs successive divisions in the space of predictor variables.

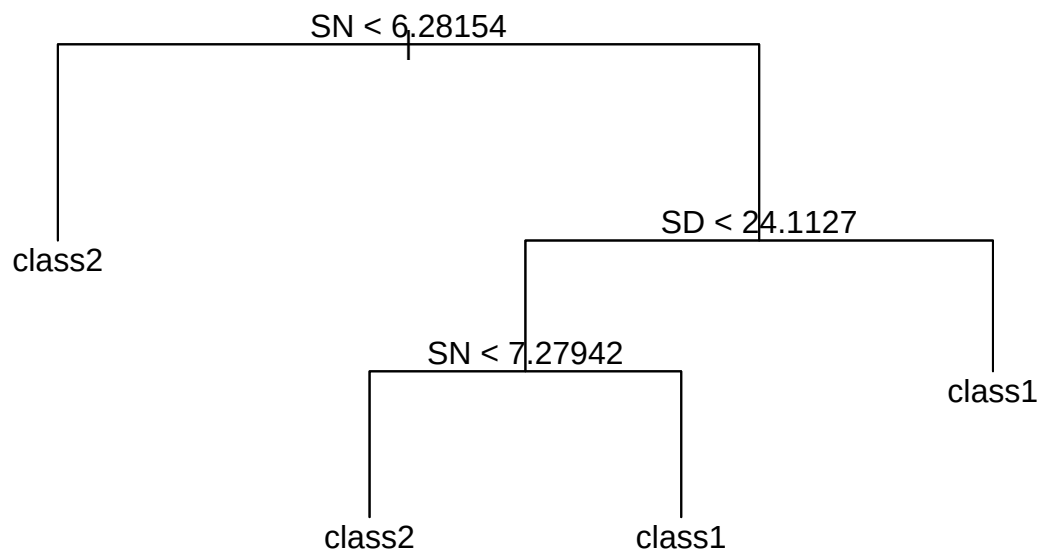
```
library(tree)

# Adjusting the decision tree to the training data
tree.dados <- tree(TSH_class ~ SN + SD + SH + R + G + B + VARI + GLI + NGRDI,
  data = dados,
  subset = l_treino)

# Summary of the adjustment (Checking which variables were used in the nodes)
summary(tree.dados)
```

```
##
## Classification tree:
## tree(formula = TSH_class ~ SN + SD + SH + R + G + B + VARI +
##       GLI + NGRDI, data = dados, subset = l_treino)
## Variables actually used in tree construction:
## [1] "SN" "SD"
## Number of terminal nodes: 4
## Residual mean deviance: 0.2208 = 5.742 / 26
## Misclassification error rate: 0.03333 = 1 / 30
```

```
# Visualizing the tree graphically
plot(tree.dados)
text(tree.dados, pretty = 0) # Add labels to nodes
```



```
# Evaluating the Tree on the Test Set
tree.pred <- predict(tree.dados, dados[-l_treino, ], type = "class")
table(Predito = tree.pred, Real = dados$TSH_class[-l_treino])
```

```
##           Real
## Predito  class2 class1
##   class2     12     0
##   class1      5    13
```

```
erro_test_tree <- mean(tree.pred != dados$TSH_class[-l_treino])
cat("Tree Error Rate in Testing:", erro_test_tree * 100, "%\n")
```

```
## Tree Error Rate in Testing: 16.66667 %
```

We performed cross-validation (cv.tree) to check if a smaller tree would improve the classification error rate. The process indicated that the original tree already has the ideal size, and there is no need for pruning.

2.7 Random Forest

To see if we can improve the predictive capacity of the simple tree, we adjusted a Random Forest model. This method creates multiple trees using resampling of the training data (Bootstrap) and random selection of variables, reducing the variance of the model.

```
library(randomForest)

set.seed(1)
# Adjusting the Random Forest with variable importance calculation (importance = TRUE)
rf.dados <- randomForest(TSH_class ~ SN + SD + SH + R + G + B + VARI + GLI + NGRDI,
                        data = dados,
                        subset = 1_treino,
                        importance = TRUE)

# Checking the importance of each variable for the model.
importance(rf.dados)
```

##		class2	class1	MeanDecreaseAccuracy	MeanDecreaseGini
##	SN	6.3284845	12.9817068	11.9267561	3.3382772
##	SD	-0.7775307	6.3895109	4.5271517	1.3623085
##	SH	2.5803703	6.3784087	5.8247557	2.0901967
##	R	-1.3159215	-1.1643016	-1.6092527	0.7474390
##	G	-0.2174371	0.2923945	0.1815363	0.6919268
##	B	-0.4108030	1.4431471	0.7437896	0.8480092
##	VARI	0.9822646	6.0003033	4.5385992	2.2433188
##	GLI	1.4521863	5.2629251	4.8205977	1.5144082
##	NGRDI	3.4937452	1.9701797	3.1019575	1.4407823

```
# Evaluating the Random Forest on the Test Set
rf.pred <- predict(rf.dados, newdata = dados[-1_treino, ])
table(Predito = rf.pred, Real = dados$TSH_class[-1_treino])
```

##		Real	
##	Predito	class2	class1
##	class2	11	3
##	class1	6	10

```
erro_test_rf <- mean(rf.pred != dados$TSH_class[-1_treino])
cat("Error Rate of Random Forest in the Test:", erro_test_rf * 100, "%\n")
```

```
## Error Rate of Random Forest in the Test: 30 %
```

2.8 General Comparison of Methodologies

Finally, we compiled the error rates of all models evaluated in the test set to determine the best classification approach for this dataset.

```
# Organizing the percentage results
erro_logistica <- erro_teste_l_glm * 100
erro_lda <- erro_teste_lda * 100
```

```

erro_knn      <- taxa_test_K3 * 100      # Assuming K=3 as the best KNN
erro_arvore   <- erro_test_tree * 100
erro_rf       <- erro_test_rf * 100

# Building the comparison table
tabela <- data.frame(
  Methodology = c("Logistic Regression", "LDA (Linear)", "KNN (K=3)", "Simple Tree", "Random Forest"),
  Boundary = c("Linear", "Linear", "Non-Linear", "Non-Linear", "Non-Linear"),
  Erro_Testes = c(erro_logistica, erro_lda, erro_knn, erro_arvore, erro_rf)
)

# Sorting from smallest to largest error
tabela_ordenada <- tabela[order(tabela$Erro_Testes), ]
print(tabela_ordenada, row.names = FALSE)

```

```

##      Methodology  Boundary Erro_Testes
## Logistic Regression   Linear    16.66667
##      Simple Tree Non-Linear    16.66667
##      LDA (Linear)   Linear    23.33333
##      KNN (K=3) Non-Linear    30.00000
##      Random Forest Non-Linear    30.00000

```

When evaluating the Logistic Regression model, we observed a lack of convergence in the algorithm and a zero training error rate. The reason for this phenomenon is directly related to the small size of the training sample compared to the large number of predictor variables, which allowed for perfect separation of classes by predictors, configuring a clear case of overfitting. Since Linear Discriminant Analysis (LDA) does not suffer from this instability in parameter estimation when classes are well separated, we chose to use it, among the linear models, for the evaluation in the third stage.

Regarding the choice of the Decision Tree over the Random Forest model, we observed that the test error rate of the Random Forest was higher than that of the simple tree. This result can be explained by the small number of observations in the training set, $n=30$. Since Random Forest uses the bootstrap process, the dozens of trees generated by the algorithm were being fitted with an even smaller number of observations in each iteration, which resulted in weak individual classifiers and impaired the accuracy of the aggregate model for this specific case. Additionally, when evaluating the variable importance graph of Random Forest, we noticed that the two most important predictors were exactly the same as those used by the decision tree in its natural construction. Associating this variable redundancy with the sample size restriction, we opted to choose the decision tree to evaluate its performance with a reduced number of variables.

2.9 Evaluating the model improvement with the removal of variables

2.9.1 Lasso

From the correlation matrix, we see that the high linear correlation between several of the explanatory variables, namely: R, G, B, VARI, GLI, and NGRDI. With the goal of evaluating whether reducing the number of variables results in improved predictive performance of the chosen models, LDA and selection tree, we will use Lasso to choose which variables should be removed.

```

library(glmnet)

all_predictors <- c("SN", "SD", "SH", "R", "G", "B", "VARI", "GLI", "NGRDI")

```

```

X_train_lasso <- as.matrix(treino[, all_predictors])
y_train_lasso <- treino$TSH_class

set.seed(1)
cv.out <- cv.glmnet(X_train_lasso, y_train_lasso, family = "binomial", alpha = 1)

lasso_coefs <- predict(cv.out, type = "coefficients", s = "lambda.1se")
print(lasso_coefs)

```

```

## 10 x 1 sparse Matrix of class "dgCMatrix"
##               lambda.1se
## (Intercept) -40.80593378
## SN          1.94705096
## SD          0.75978414
## SH          3.09579171
## R            .
## G          0.02574245
## B            .
## VARI         .
## GLI          .
## NGRDI        .

```

Note that the Lasso method excluded, with the exception of the variables SN, SD, SH, and G, the coefficients associated with the other variables. Thus, we will proceed with the adjustment of the LDA and selection tree models using these variables.

```

selected_vars <- rownames(lasso_coefs)[which(lasso_coefs != 0)]
selected_vars <- selected_vars[selected_vars != "(Intercept)"]
print(selected_vars)

```

```
## [1] "SN" "SD" "SH" "G"
```

```

#-----
dados_reduzidos <- dados[, c(selected_vars, "TSH_class")]
treino_reduzido <- dados_reduzidos[1_treino, ]
teste_reduzido  <- dados_reduzidos[-1_treino, ]

```

2.9.2 Linear Discriminant Analysis (LDA)

```

lda.fit_reduzido <- lda(TSH_class ~ SN + SD + SH + G , data = treino_reduzido)
print(lda.fit_reduzido)

```

```

## Call:
## lda(TSH_class ~ SN + SD + SH + G, data = treino_reduzido)
##
## Prior probabilities of groups:
##   class2   class1
## 0.4333333 0.5666667
##

```

```
## Group means:
##          SN          SD          SH          G
## class2 6.425940 23.70295 2.051285 86.06916
## class1 7.534298 24.68713 2.258829 85.39781
##
## Coefficients of linear discriminants:
##          LD1
## SN 1.6081691
## SD 0.7640286
## SH 3.9132623
## G  0.2013135
```

```
#
# (*)TESTING ERROR RATE -----
lda.pred_reduzido <- predict(lda.fit_reduzido, newdata = teste_reduzido)
lda.class_reduzido <- lda.pred_reduzido$class

cat("\n--- LDA (REDUCED MODEL) TEST CONFUSION MATRIX ---\n")
```

```
##
## --- LDA (REDUCED MODEL) TEST CONFUSION MATRIX ---
```

```
print(table(Predicted = lda.class_reduzido, True = teste_reduzido$TSH_class))
```

```
##          True
## Predicted class2 class1
##   class2      11      1
##   class1       6     12
```

```
erro_test_lda_reduzido <- mean(lda.class_reduzido != teste_reduzido$TSH_class)
cat("LDA (Reduced Model) Test Error Rate:", erro_test_lda_reduzido * 100, "%\n")
```

```
## LDA (Reduced Model) Test Error Rate: 23.33333 %
```

TRAINING ERROR RATE

```
lda.pred_treino_reduzido <- predict(lda.fit_reduzido, newdata = treino_reduzido)
lda.class_treino_reduzido <- lda.pred_treino_reduzido$class
cat("\n--- LDA (REDUCED MODEL) TRAINING CONFUSION MATRIX ---\n")
```

```
##
## --- LDA (REDUCED MODEL) TRAINING CONFUSION MATRIX ---
```

```
print(table(Predicted = lda.class_treino_reduzido, Real = treino_reduzido$TSH_class))
```

```
##          Real
## Predicted class2 class1
##   class2      12      0
##   class1       1     17
```

```
erro_treino_lda_reduzido <- mean(lda.class_treino_reduzido != treino_reduzido$TSH_class)
cat("LDA (Reduced Model) Training Error Rate:", erro_treino_lda_reduzido * 100, "%\n")
```

```
## LDA (Reduced Model) Training Error Rate: 3.333333 %
```

```
##(*) FULL DATASET ERROR RATE -----
# Refitting the LDA model across the entire reduced dataset
lda.fit_total_reduzido <- lda(TSH_class ~ SN + SD + SH + G,
                             data = dados_reduzidos)

lda.pred_total_reduzido <- predict(lda.fit_total_reduzido, newdata = dados_reduzidos)
lda.class_total_reduzido <- lda.pred_total_reduzido$class

# Full Dataset Confusion Matrix
cat("\n--- LDA (REDUCED MODEL) FULL DATASET CONFUSION MATRIX ---\n")
```

```
##
## --- LDA (REDUCED MODEL) FULL DATASET CONFUSION MATRIX ---
```

```
print(table(Predicted = lda.class_total_reduzido, Real = dados_reduzidos$TSH_class))
```

```
##           Real
## Predicted class2 class1
##   class2      26      2
##   class1       4     28
```

```
# Full Dataset Error Calculation
erro_total_lda_reduzido <- mean(lda.class_total_reduzido != dados_reduzidos$TSH_class)
cat("LDA (Reduced Model) Full Dataset Error Rate:", erro_total_lda_reduzido * 100, "%\n")
```

```
## LDA (Reduced Model) Full Dataset Error Rate: 10 %
```

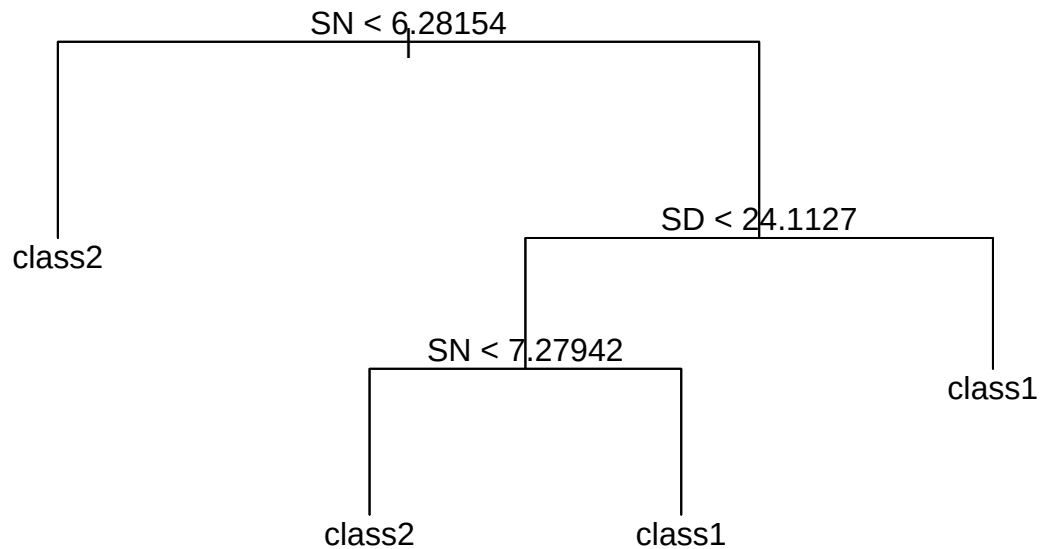
2.9.3 Simple Decision Tree

```
library(tree)
tree.dados_redu <- tree(TSH_class ~ SN + SD + SH + G,
                       data = dados_reduzidos,
                       subset = l_treino)

summary(tree.dados_redu)
```

```
##
## Classification tree:
## tree(formula = TSH_class ~ SN + SD + SH + G, data = dados_reduzidos,
##       subset = l_treino)
## Variables actually used in tree construction:
## [1] "SN" "SD"
## Number of terminal nodes: 4
## Residual mean deviance: 0.2208 = 5.742 / 26
## Misclassification error rate: 0.03333 = 1 / 30
```

```
#
# Visualizing the Tree Structure
plot(tree.dados_redu)
text(tree.dados_redu, pretty = 0) # Displays node splits with actual threshold values
```



```
# TESTING ERROR RATE -----
# Predicting classes on the independent test subset
tree.pred_redu <- predict(tree.dados_redu, newdata = dados_reduzidos[-l_treino, ], type = "class")
# Testing Confusion Matrix
cat("\n--- DECISION TREE TESTING CONFUSION MATRIX---\n")
```

```
##
## --- DECISION TREE TESTING CONFUSION MATRIX---
```

```
print(table(Predicted = tree.pred_redu, True = dados_reduzidos$TSH_class[-l_treino]))
```

```
##          True
## Predicted class2 class1
##    class2      12      0
##    class1       5     13
```

```
#Testing Error Calculation
erro_test_tree_redu <- mean(tree.pred != dados_reduzidos$TSH_class[-l_treino])
cat("Decision Tree Testing Error Rate:", erro_test_tree_redu * 100, "%\n")
```

```
## Decision Tree Testing Error Rate: 16.66667 %
```

TRAINF ERROR RATE

```
#predicting classes on the train data set
tree.pred_train_redu <- predict(tree.dados_redu, newdata = dados_reduzidos[l_treino, ], type = "class")
# Traing Confusion Matrix
cat("\n--- DECISION TREE TRAING CONFUSION MATRIX ---\n")
```

```
##
## --- DECISION TREE TRAING CONFUSION MATRIX ---
```

```
print(table(Predicted = tree.pred_train_redu, True = dados_reduzidos$TSH_class[l_treino]))
```

```
##           True
## Predicted class2 class1
##   class2      12      0
##   class1       1     17
```

```
# Testing Error Calculation
erro_train_tree_redu <- mean(tree.pred_train_redu != dados_reduzidos$TSH_class[l_treino])
cat("Decision Tree Traing Error Rate:", erro_train_tree_redu * 100, "%\n")
```

```
## Decision Tree Traing Error Rate: 3.333333 %
```

FULL DATASET ERROR RATE

```
#Refitting the decision tree across the entire dataset
tree.dados_total_redu <- tree(TSH_class ~ SN + SD + SH + G,
                             data = dados_reduzidos)

#Predicting classes.
tree.pred_total_redu <- predict(tree.dados_total_redu, newdata = dados_reduzidos, type = "class")

#Confusion Matrix
cat("\n--- DECISION TREE FULL DATASET CONFUSION MATRIX ---\n")
```

```
##
## --- DECISION TREE FULL DATASET CONFUSION MATRIX ---
```

```
print(table(Predicted = tree.pred_total_redu, True = dados_reduzidos$TSH_class))
```

```
##           True
## Predicted class2 class1
##   class2      30      2
##   class1       0     28
```

```
#Error rate
erro_total_tree_redu <- mean(tree.pred_total_redu != dados_reduzidos$TSH_class)
cat("Decision Tree Full Dataset Error Rate:", erro_total_tree_redu * 100, "%\n")
```

```
## Decision Tree Full Dataset Error Rate: 3.333333 %
```

2.9.4 Table for comparing the models

```
final_table <- data.frame(
  Approach = c("Full Model ",
               "Lasso Reduced Model "),
  LDA_Test_Error = c(erro_teste_lda, erro_test_lda_reduzido),
  Decision_Tree_Test_Error = c(erro_test_tree, erro_test_tree_redu)
)

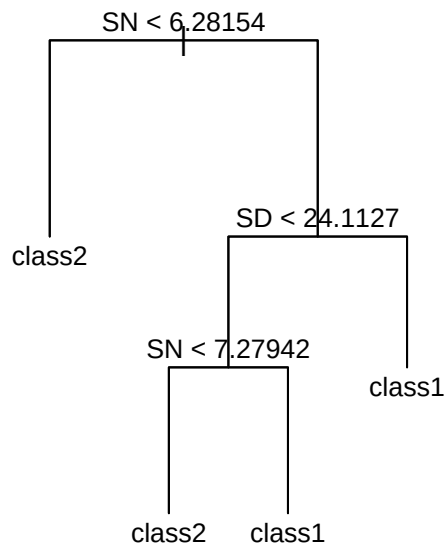
#

print(final_table, row.names = FALSE)
```

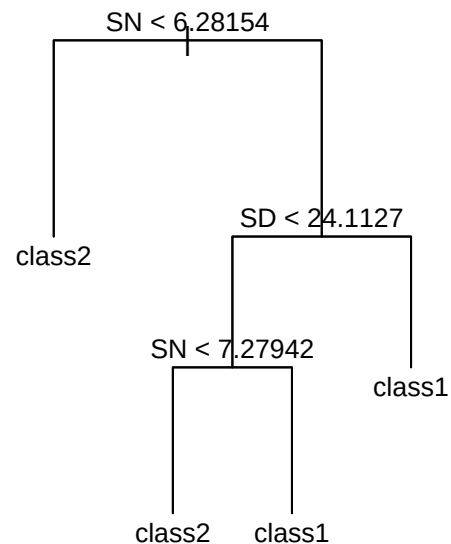
```
##           Approach LDA_Test_Error Decision_Tree_Test_Error
##           Full Model      0.2333333      0.1666667
## Lasso Reduced Model      0.2333333      0.1666667
```

```
# arvores
par(mfrow = c(1, 2))
plot(tree.dados)
text(tree.dados, pretty = 0, cex = 0.8)
title(main = "Decision Tree ( 9 Variables)", line = 2, cex.main = 1)
#
plot(tree.dados_redu)
text(tree.dados_redu, pretty = 0, cex = 0.8)
title(main = "Decision Tree ( 4 Variables)", line = 2, cex.main = 1)
```

Decision Tree (9 Variables)



Decision Tree (4 Variables)



```
par(mfrow = c(1, 1))
```

We observed that the test error rates remained the same when evaluating the LDA model and the decision tree with a reduced number of predictors. This fact highlights the redundancy of the removed variables, since their exclusion did not affect the predictive capacity and the fit of the models. In the case of LDA, the removal of these variables was fundamental, as they did not contribute new information to the classification and also harmed the accuracy of the coefficient estimates, inflating their associated variances.

With regard to the decision tree, it was expected that the structure and estimates would remain unchanged, since the recursive binary splitting mechanics of the algorithm naturally make it immune to the problem of multicollinearity.

3 Regression Problem

3.1 Regression Models

In the first step, we converted the TSH variable to continuous, as it was used as a factor in the classification models part, and then we divided the data into a 50% training and 50% test validation set.

```
dados_modelagem <- dados %>%  
  dplyr::select(-Family) %>%  
  dplyr::mutate(TSH = as.numeric(as.character(TSH)))  
  
# Preparation and Division (50/50)
```

```

# Removing the Family variable
dados_modelagem <- dados %>% dplyr::select(-Family)

#Reproducible division of data
set.seed(123)
indices_treino <- sample(1:nrow(dados_modelagem), size = 0.5 * nrow(dados_modelagem))

treino <- dados_modelagem[indices_treino, ]
teste  <- dados_modelagem[-indices_treino, ]

```

We use a Linear model with all variables

```

modelo_completo <- lm(TSH ~ . - Family, data = dados)
summary(modelo_completo)

```

```

##
## Call:
## lm(formula = TSH ~ . - Family, data = dados)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -11.1587  -4.2639  -0.4446   4.0890  14.3656
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -203.8704     61.1177  -3.336 0.001629 **
## SN              11.8754      1.1838  10.032 1.81e-13 ***
## SD              5.8444      0.7853   7.442 1.37e-09 ***
## SH             26.2396      5.8202   4.508 4.08e-05 ***
## R               1.1356      6.2154   0.183 0.855780
## G             -14.8871      9.3962  -1.584 0.119541
## B              14.2539     14.6473   0.973 0.335261
## VARI           1469.9673    2338.0273   0.629 0.532452
## GLI            4200.4752    4841.1128   0.868 0.389806
## NGRDI          -4565.6409    7653.3290  -0.597 0.553550
## TSH_classclass1  9.6429      2.5682   3.755 0.000461 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.198 on 49 degrees of freedom
## Multiple R-squared:  0.937, Adjusted R-squared:  0.9242
## F-statistic: 72.94 on 10 and 49 DF, p-value: < 2.2e-16

```

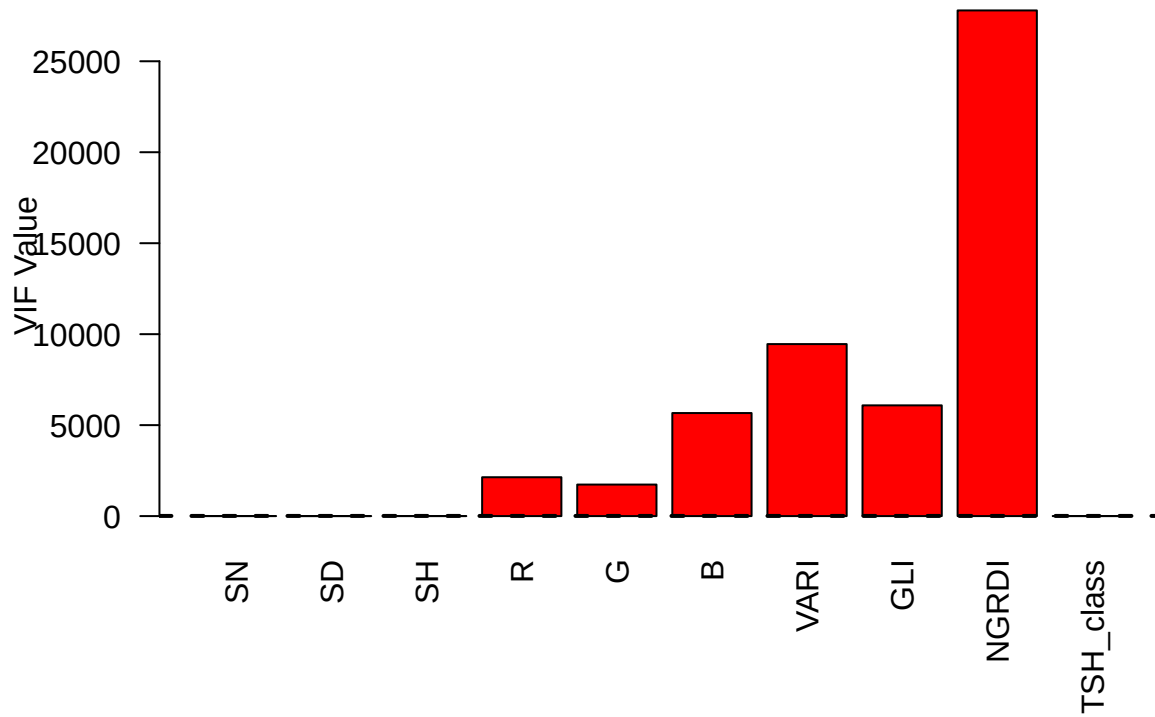
```

# Calculation and plotting of the VIF
valores_vif <- vif(modelo_completo)

barplot(valores_vif,
        main = "Variance Inflation Factor (VIF)",
        col = ifelse(valores_vif > 10, "red", "steelblue"),
        las = 2,
        ylab = "VIF Value")
abline(h = 10, col = "black", lty = 2, lwd = 2)

```

Variance Inflation Factor (VIF)



Due to the multicollinearity problem, initially indicated by the correlation matrix and confirmed by the high VIF values, we performed a variable selection process in the model in order to remove the multicollinearity and obtain a more parsimonious model. For this, a stepwise selection process was carried out.

```
# Selection Stepwise Forward
mod_step_vs <- step(lm(TSH ~ 1, data = treino),
                    scope = formula(lm(TSH ~ ., data = treino)),
                    direction = "both", trace = 0)
summary(mod_step_vs)
```

```
##
## Call:
## lm(formula = TSH ~ TSH_class + SH + SN + SD, data = treino)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.7498  -4.5886   0.7549   4.4035  13.6836
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -182.336     28.869  -6.316 1.31e-06 ***
## TSH_classclass1    7.341      4.052   1.812 0.082070 .
## SH             35.568      9.692   3.670 0.001150 **
## SN             12.617      1.580   7.987 2.42e-08 ***
## SD              5.467      1.297   4.214 0.000285 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.4 on 25 degrees of freedom
## Multiple R-squared:  0.9296, Adjusted R-squared:  0.9183
## F-statistic: 82.52 on 4 and 25 DF,  p-value: 4.965e-14
```

3.2 Principal Component Extraction (PCA)

As an alternative to the multicollinearity of the linear model, dimensionality reduction using principal components emerges as an excellent option, since the components are orthogonal to each other and do not exhibit autocorrelation.

Therefore, we chose to test this approach, as it does not require the exclusion of variables in this case.

```
# --- Dimensionality Reduction (PCA) and Visualization ---
library(ggplot2)
library(factoextra)

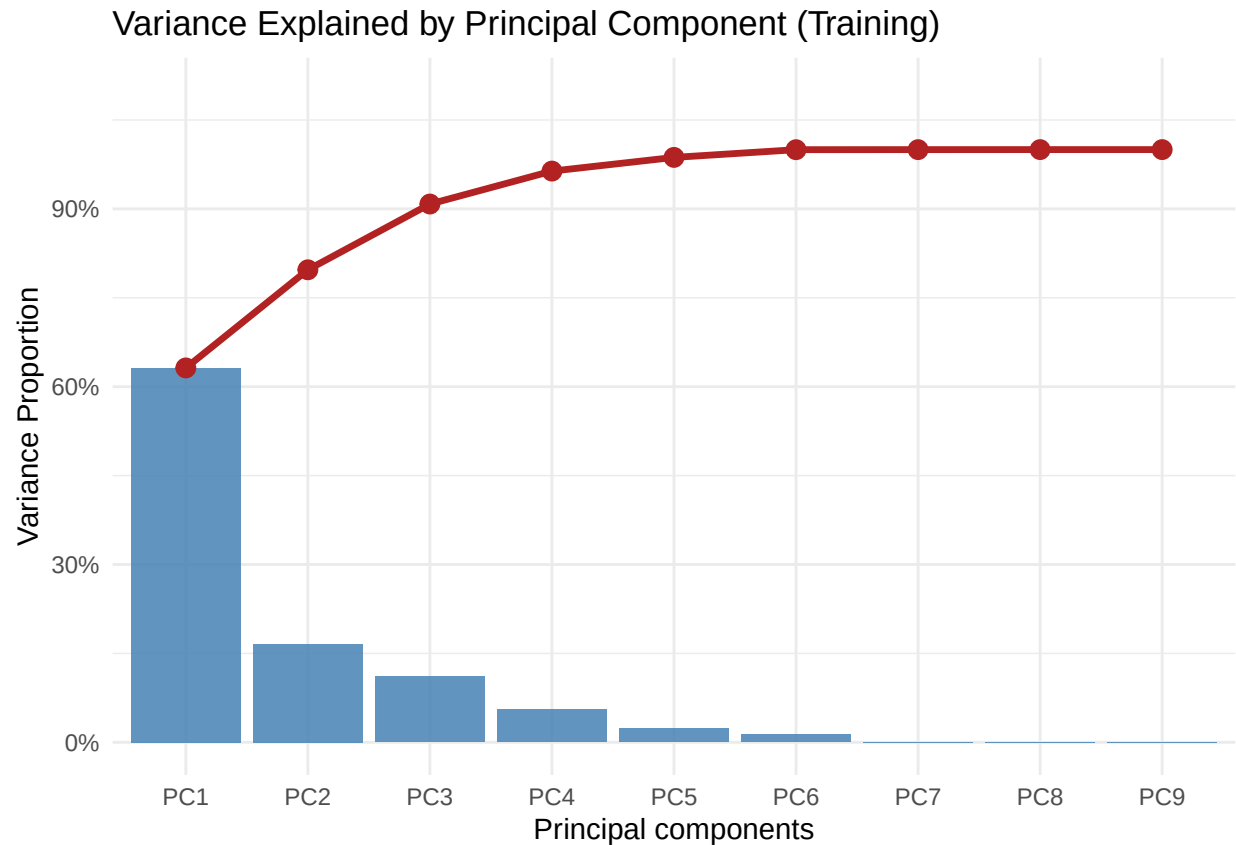
# 1. Extracting PCA only in Training
dados_pca_treino_covariadas <- treino %>%
  dplyr::select(-TSH) %>%
  dplyr::select(where(is.numeric)) # Safety lock for numeric variables

pca_treino <- prcomp(dados_pca_treino_covariadas, center = TRUE, scale. = TRUE)

# 2. Preparing the data for the Scree Plot
var_explicada <- pca_treino$sdev^2
prop_var <- var_explicada / sum(var_explicada)
var_acumulada <- cumsum(prop_var)

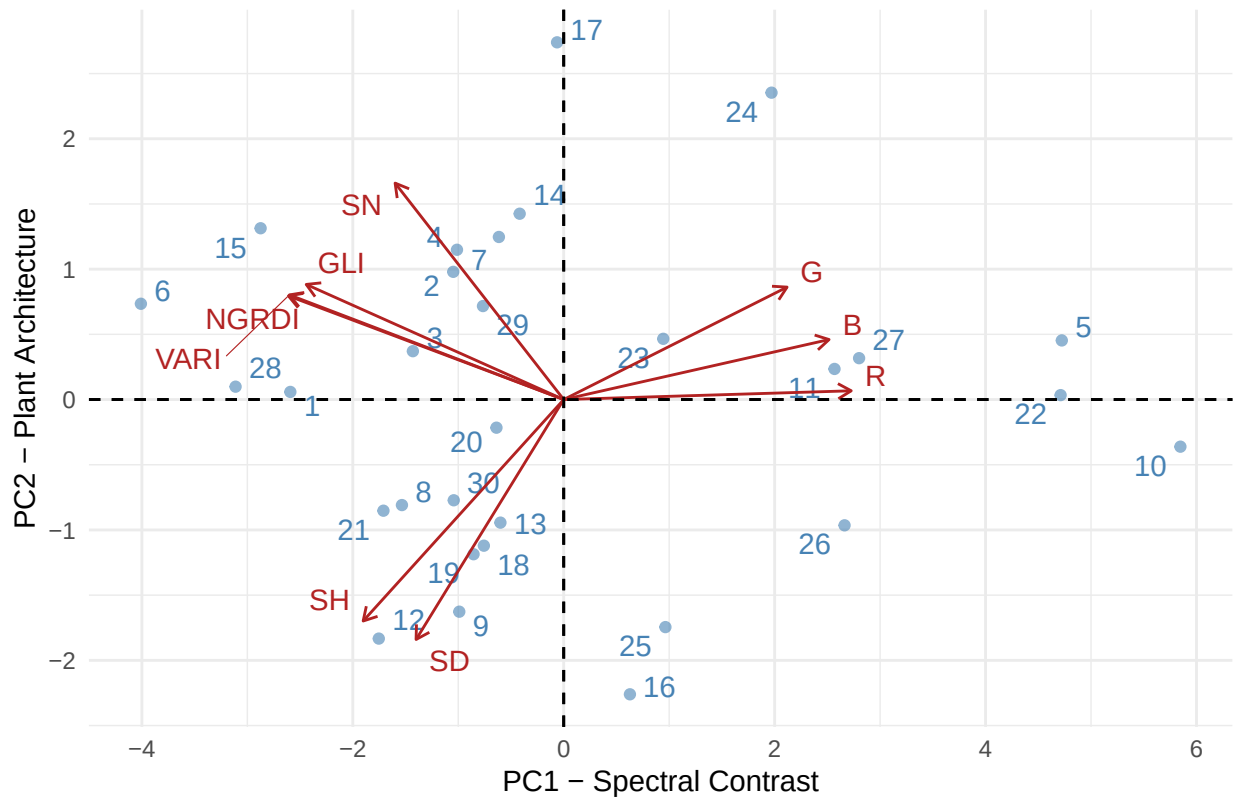
df_pca_var <- data.frame(
  Componente = factor(paste0("PC", 1:length(prop_var)), levels = paste0("PC", 1:length(prop_var))),
  Proporcão = prop_var,
  Acumulada = var_acumulada
)

# 3. Variance Plot (Scree Plot)
ggplot(df_pca_var, aes(x = Componente)) +
  geom_bar(aes(y = Proporcão), stat = "identity", fill = "steelblue", alpha = 0.85) +
  geom_line(aes(y = Acumulada, group = 1), color = "firebrick", linewidth = 1.2) +
  geom_point(aes(y = Acumulada), color = "firebrick", size = 3) +
  scale_y_continuous(labels = scales::percent_format(), limits = c(0, 1.1)) +
  theme_minimal() +
  labs(title = "Variance Explained by Principal Component (Training)",
       x = "Principal components", y = "Variance Proportion")
```



```
# 4. Biplot Graph
fviz_pca_biplot(pca_treino,
  repel = TRUE,
  col.ind = "steelblue",
  alpha.ind = 0.6,
  title = "PCA Biplot: Dispersion and Projection of Variables (Training)",
  xlab = "PC1 - Spectral Contrast",
  ylab = "PC2 - Plant Architecture") +
  theme_minimal() +
  theme(plot.title = element_text(face = "bold", hjust = 0.5))
```

PCA Biplot: Dispersion and Projection of Variables (Training)



5. Projecting the test data into the training PCA space for future use

```
dados_pca_teste_covariadas <- teste %>%
```

```
  dplyr::select(-TSH) %>%
```

```
  dplyr::select(where(is.numeric))
```

```
pred_pca_teste <- as.data.frame(predict(pca_treino, newdata = dados_pca_teste_covariadas)[, 1:4])
```

It was decided to retain the first four components, which account for most of the variability of the original predictors.

3.3 Adjustment of the principal component regression model

Creating the PCR database for training (using the first 3 PCs)

```
dados_pcr_treino <- cbind(treino %>% dplyr::select(TSH), pca_treino$x[, 1:4])
```

Complete PCR Model (3 components)

```
modelo_pcr_completo <- lm(TSH ~ PC1 + PC2 + PC3 + PC4, data = dados_pcr_treino)
```

```
summary(modelo_pcr_completo)
```

```
##
```

```
## Call:
```

```
## lm(formula = TSH ~ PC1 + PC2 + PC3 + PC4, data = dados_pcr_treino)
```

```
##
```

```
## Residuals:
##      Min       1Q   Median       3Q      Max
## -17.4538  -4.7851   0.1795   4.6519  13.8477
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 116.8254      1.4147  82.579  < 2e-16 ***
## PC1          -8.5877      0.6035 -14.230 1.71e-13 ***
## PC2          -1.7140      1.1787  -1.454   0.158
## PC3          -6.6838      1.4402  -4.641 9.45e-05 ***
## PC4         -17.3368      2.0336  -8.525 7.23e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.749 on 25 degrees of freedom
## Multiple R-squared:  0.9228, Adjusted R-squared:  0.9104
## F-statistic: 74.7 on 4 and 25 DF,  p-value: 1.562e-13
```

```
# Stepwise refinement in the PCR model (Direction: Both)
modelo_pcr_refinado <- step(modelo_pcr_completo, direction = "both", trace = 0)
summary(modelo_pcr_refinado)
```

```
##
## Call:
## lm(formula = TSH ~ PC1 + PC2 + PC3 + PC4, data = dados_pcr_treino)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -17.4538  -4.7851   0.1795   4.6519  13.8477
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 116.8254      1.4147  82.579  < 2e-16 ***
## PC1          -8.5877      0.6035 -14.230 1.71e-13 ***
## PC2          -1.7140      1.1787  -1.454   0.158
## PC3          -6.6838      1.4402  -4.641 9.45e-05 ***
## PC4         -17.3368      2.0336  -8.525 7.23e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.749 on 25 degrees of freedom
## Multiple R-squared:  0.9228, Adjusted R-squared:  0.9104
## F-statistic: 74.7 on 4 and 25 DF,  p-value: 1.562e-13
```

Even though it wasn't significant, after performing the variable selection process, the PC2 component remained in the model, so we chose to keep it.

3.4 Partial Least Squares (PLS) Model

```
library(pls)
```

```

# Ensuring that the PLS database in training contains only numerical variables.
dados_pls_treino <- treino %>% dplyr::select(where(is.numeric))

# 1. Initial PLS adjustment with Cross-Validation (CV) on the Training set
set.seed(1)
pls.fit <- plsr(TSH ~ ., data = dados_pls_treino, scale = TRUE, validation = "CV")

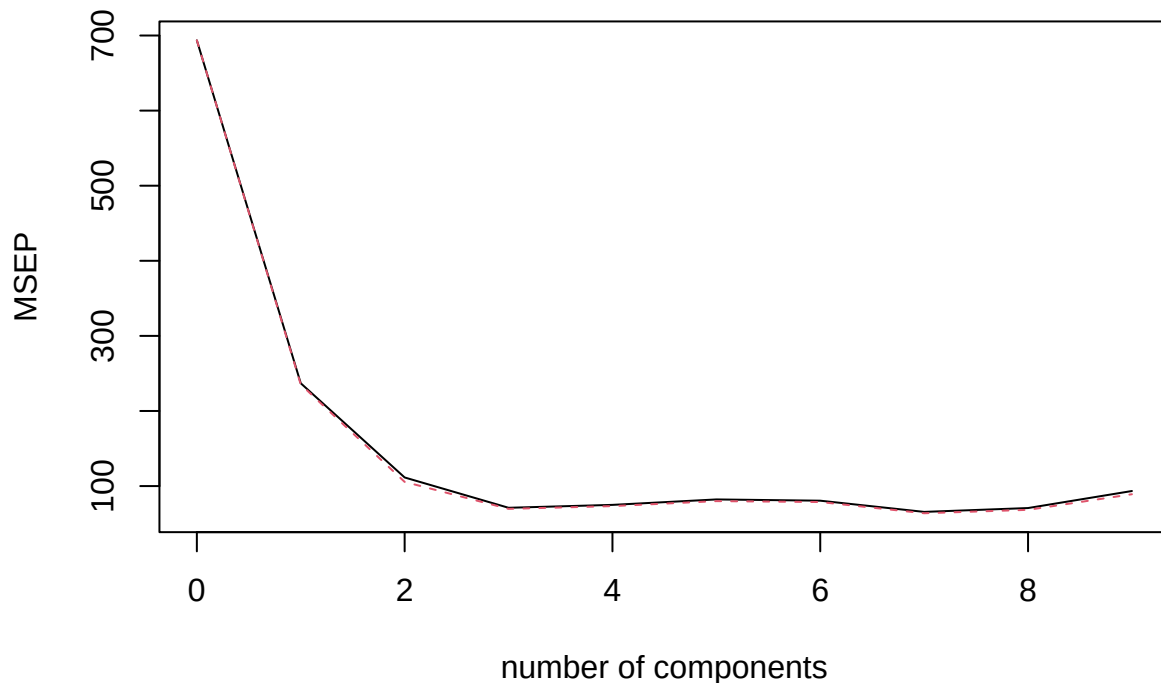
# Displays the model summary (cumulative R2 and error per component)
summary(pls.fit)

## Data:      X dimension: 30 9
## Y dimension: 30 1
## Fit method: kernelppls
## Number of components considered: 9
##
## VALIDATION: RMSEP
## Cross-validated using 10 random segments.
##      (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## CV           26.34   15.39   10.56   8.432   8.659   9.067   8.974
## adjCV        26.34   15.34   10.28   8.344   8.564   8.942   8.874
##      7 comps  8 comps  9 comps
## CV          8.101   8.411   9.666
## adjCV       7.989   8.267   9.441
##
## TRAINING: % variance explained
##      1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps  8 comps
## X          62.80   72.08   82.78   96.09   98.39   99.98  100.00  100.00
## TSH        68.92   89.83   92.59   92.84   93.00   93.05   94.82   95.06
##      9 comps
## X          100.00
## TSH         95.14

# Plot the MSEP (Mean Squared Error of Prediction) to identify the minimum point.
validationplot(pls.fit, val.type = "MSEP", main = "Prediction Error (MSEP) by Component (Training)")

```

Prediction Error (MSEP) by Component (Training)



After analyzing the graph, we can see that the MSEP stops falling from the point corresponding to the fourth component.

```
# 2. Final Model Adjustment with the Ideal Number of Components (3 components)
# We will use the name mod_pls_vs to connect with the subsequent prediction block.
mod_pls_vs <- plsr(TSH ~ ., data = dados_pls_treino, scale = TRUE, ncomp = 3)

summary(mod_pls_vs)
```

```
## Data:      X dimension: 30 9
## Y dimension: 30 1
## Fit method: kernelpls
## Number of components considered: 3
## TRAINING: % variance explained
##      1 comps  2 comps  3 comps
## X      62.80   72.08   82.78
## TSH     68.92   89.83   92.59
```

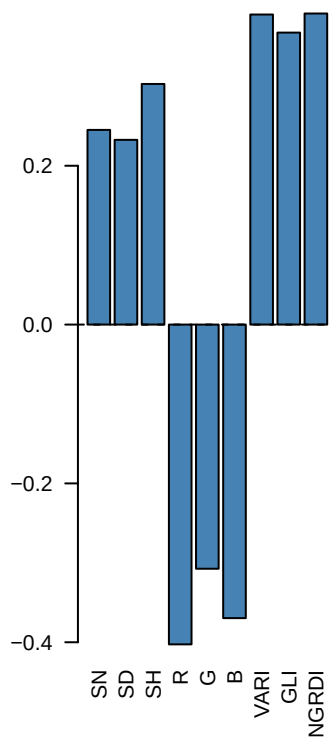
The final Partial Least Squares (PLS) fitted model with 3 components demonstrated excellent predictive capacity and high efficiency in dimensionality reduction. The three constructed latent variables were able to synthesize 82.78% of the variability contained in the original set of predictors (X), eliminating the redundancies caused by the severe multicollinearity observed previously.

```
# 3. Extract the loadings of the predictor variables for the 3 components.
cargas_pls <- loadings(mod_pls_vs)[, 1:3]
print(cargas_pls)
```

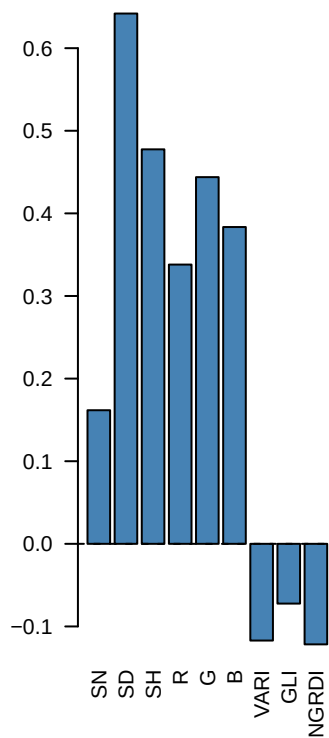
```
##          Comp 1      Comp 2      Comp 3
## SN      0.2451264  0.16172842  0.8336210170
## SD      0.2325949  0.64192788 -0.5824545807
## SH      0.3029762  0.47753399 -0.2653559780
## R       -0.4026623  0.33803758 -0.0445310720
## G       -0.3074242  0.44389433 -0.0570787146
## B       -0.3696373  0.38348109  0.0116613168
## VARI     0.3904206 -0.11706831 -0.0002752092
## GLI      0.3676526 -0.07232953 -0.0564609700
## NGRDI    0.3916718 -0.12170056  0.0031341860
```

```
# Bar chart to visualize the weights of each component
par(mfrow = c(1, 3))
for(i in 1:3) {
  barplot(cargas_pls[, i],
    main = paste("weights of each component", i),
    col = "steelblue", las = 2)
  abline(h = 0, lty = 2)
}
```

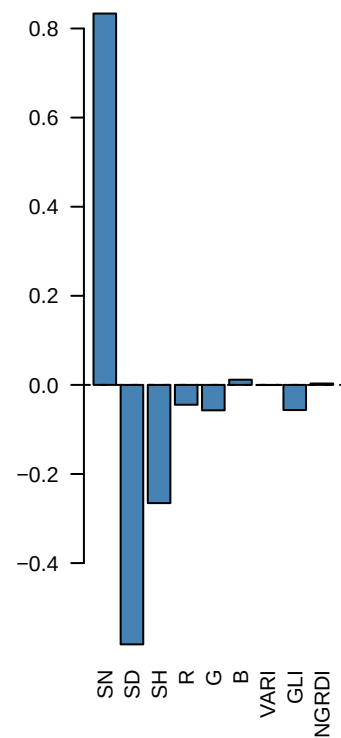
weights of each component 1



weights of each component 2



weights of each component 3



```
# Reset the graphic layout so it doesn't affect future plots in the document.
par(mfrow = c(1, 1))
```

The interpretation of the loadings of the latent variables brought very interesting nuances to the structure of the PLS model. Component 1 was characterized by the strong contrast between the positive spectral indices (NGRDI = 0.392; VARI = 0.390) and the negative visible bands (R = -0.403; B = -0.370), also associated with plant height growth (SH = 0.303). Component 2, responsible for the large leap in explaining the variance of TSH, assumed the role of the axis of structural robustness. In it, the biometric variables of stem diameter (SD = 0.642) and height (SH = 0.478) took center stage, demonstrating that the physical support structure is determinant for the final yield. Finally, Component 3 revealed the PLS's ability to model the morphological compensation architecture of the crop: it exhibits a very strong positive weight for the number of stalks (SN = 0.834) counteracting the negative weights of diameter (SD = -0.582) and height (SH = -0.265), perfectly capturing the balance between many thin stalks versus few thick stalks in the final productivity adjustment.

3.5 Predictions and Final Comparison of Regression Models

With all models fitted to the training set, the final step consists of applying the prediction rules of each methodology to the validation (test) set and comparing their respective error metrics.

```
mod_completo_vs <- lm(TSH ~ ., data = treino)

# 2. Generating predictions for the test set
p_completo_vs <- predict(mod_completo_vs, newdata = teste)
p_step_vs <- predict(mod_step_vs, newdata = teste)
p_pcr_completo <- predict(modelo_pcr_completo, newdata = pred_pca_teste)
p_pcr_refinado <- predict(modelo_pcr_refinado, newdata = pred_pca_teste)
p_pls_vs <- as.vector(predict(mod_pls_vs, ncomp = 3, newdata = teste))

# 3. Isolating the actual TSH values from the test set
reais_teste <- teste$TSH

# Auxiliary function for calculating the R2 in the test.
calc_r2 <- function(real, pred) {
  1 - sum((real - pred)^2) / sum((real - mean(real))^2)
}

# 4. Building the data frame with the evaluation metrics.
metricas_regressao <- data.frame(
  Modelo = c("Full Linear", "Linear Stepwise", "Full PCR (4 Comp)", "Stepwise PCR", "PLS (3 Comp)"),
  RMSE = c(sqrt(mean((reais_teste - p_completo_vs)^2)),
            sqrt(mean((reais_teste - p_step_vs)^2)),
            sqrt(mean((reais_teste - p_pcr_completo)^2)),
            sqrt(mean((reais_teste - p_pcr_refinado)^2)),
            sqrt(mean((reais_teste - p_pls_vs)^2))),
  R2 = c(calc_r2(reais_teste, p_completo_vs),
          calc_r2(reais_teste, p_step_vs),
          calc_r2(reais_teste, p_pcr_completo),
          calc_r2(reais_teste, p_pcr_refinado),
          calc_r2(reais_teste, p_pls_vs)),
  MAE = c(mean(abs(reais_teste - p_completo_vs)),
            mean(abs(reais_teste - p_step_vs)),
```

```

    mean(abs(reais_teste - p_pcr_completo)),
    mean(abs(reais_teste - p_pcr_refinado)),
    mean(abs(reais_teste - p_pls_vs)))
)

# 5. Ordering the table by lowest RMSE (from best to worst model))
metricas_regressao <- metricas_regressao[order(metricas_regressao$RMSE), ]

# 6. Rendering the formatted table
knitr::kable(metricas_regressao, digits = 4, align = "c",
              caption = "Comparison of Regression Model Performance (Test Set)")

```

Table 1: Comparison of Regression Model Performance (Test Set)

	Modelo	RMSE	R2	MAE
2	Linear Stepwise	6.7115	0.8669	4.7851
5	PLS (3 Comp)	8.4540	0.7887	6.4513
1	Full Linear	8.8106	0.7706	6.6594
3	Full PCR (4 Comp)	9.5315	0.7315	7.0119
4	Stepwise PCR	9.5315	0.7315	7.0119

The Linear Stepwise model presented the best overall predictive metrics (lowest RMSE and highest R^2), indicating that variable selection was efficient. However, the loss of information in variable selection processes is detrimental in an informational context. In this case, the PLS model (with 3 components) proves to be a highly advantageous alternative because it retains information from all original variables, solves the multicollinearity problem by creating latent variables, and still delivers robust and satisfactory predictive performance on the test set.

3.6 Residual Analysis of the PLS Model

```

# --- Visual Normality Analysis of Residuals (PLS - 3 Comp) ---
library(ggplot2)

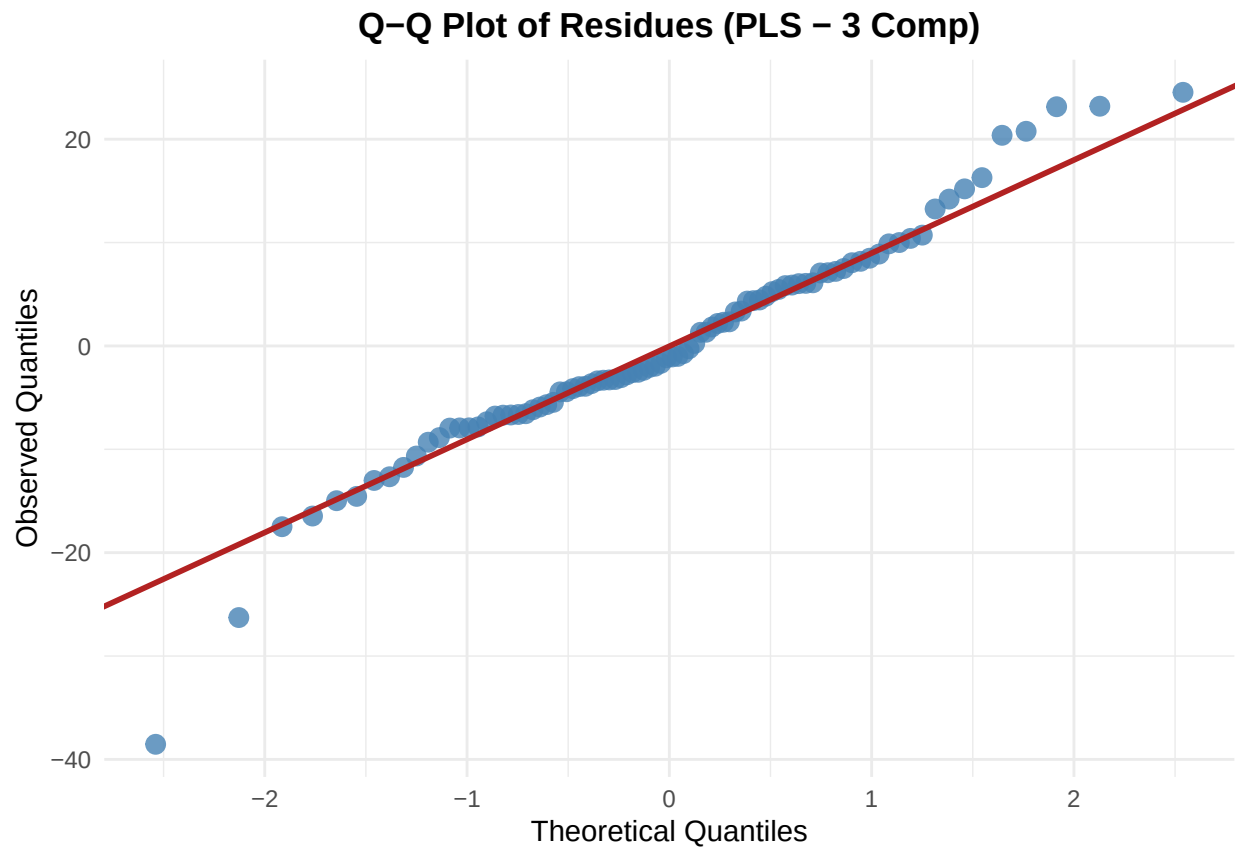
# 1. Extracting only the residues strictly for the 3 components
residuos_pls <- as.vector(residuals(mod_pls_vs, ncomp = 3))

# Creating a data frame for plotting.
df_residuos_pls <- data.frame(Residuos = residuos_pls)

# 2. Single Graph: Q-Q Plot (To check for Normality)
grafico_qq <- ggplot(df_residuos_pls, aes(sample = Residuos)) +
  stat_qq(color = "steelblue", size = 3, alpha = 0.8) +
  stat_qq_line(color = "firebrick", linewidth = 1) +
  theme_minimal() +
  labs(title = "Q-Q Plot of Residues (PLS - 3 Comp)",
       x = "Theoretical Quantiles", y = "Observed Quantiles") +
  theme(plot.title = element_text(face = "bold", hjust = 0.5))

# graph
print(grafico_qq)

```



3.7 Residual analysis of the Stepwise Linear Model

```
library(lmtest)
# 1. Shapiro-Wilk Normality Test
# Null Hypothesis (H0): The residuals have a Normal distribution
shapiro.test(residuals(mod_step_vs))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  residuals(mod_step_vs)
## W = 0.98216, p-value = 0.8797
```

```
# 2. Breusch-Pagan Homoscedasticity Test
# Null Hypothesis (H0): The variance of the residuals is constant (Homoscedasticity)
bptest(mod_step_vs)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  mod_step_vs
## BP = 6.6842, df = 4, p-value = 0.1535
```

```
# 3. Durbin-Watson Independence Test  
# Null Hypothesis (H0): There is no autocorrelation (the residuals are independent)  
dwtest(mod_step_vs)
```

```
##  
## Durbin-Watson test  
##  
## data: mod_step_vs  
## DW = 1.7733, p-value = 0.2542  
## alternative hypothesis: true autocorrelation is greater than 0
```

4 Conclusion

Based on the low test error rates and the test RSS values presented by the selected models, it is concluded that the utilization of spectral variables collected via drones, combined with the physical variables, stalk number (SN) and stalk diameter (SD), enables the screening, discarding, and identification of elite genotypes. This conclusion paves the way for a considerable improvement in logistics and labor efficiency, as it eliminates the need for manual weighing of sugarcane in genetic breeding programs.